JSONata

Query and reshape JSON with compact expressions

Install & load

```
INSTALL jsonata FROM community;
LOAD jsonata;
```

01 At a glance

Filter nested arrays, construct output objects, and aggregate values within a JSON document. JSONata keeps these transformations in one SQL expression and returns JSON that can be cast, extracted, or expanded by DuckDB.

JSON document → JSONata expression → New JSON shape

02 SQL entry point + expression syntax

`jsonata(expr, data)`

Evaluate an expression against a JSON document.

`jsonata(expr, data, bindings)`

Bind JSON object keys as $variables in the expression.

`Account.Name`

Follow nested fields using dotted paths.

`items[price > $limit]`

Filter array members by a predicate.

`{"total": $sum(prices)}`

Construct an output object with an aggregate.

`jsonata_validate(expr)`

Check expression syntax without evaluating it (DuckDB 2.0 build).

03 Keep in mind

Bind values, keep expressions stable

Pass thresholds and other parameters through the bindings object instead of concatenating them into the expression. Its keys omit the dollar sign: {"limit": 100} is read as $limit inside JSONata.

04 Filter with a bound threshold

```
SELECT jsonata(
  '[items[price > $limit].id]',
  '{"items":[{"id":"A","price":20},' ||
  '{"id":"B","price":120}]}',
  '{"limit":100}'
);
-- ["B"]
```

The outer brackets preserve an array for one matching item.

05 Build a summary object

```
SELECT jsonata(
  '{"customer": name, "total": $sum(prices)}',
  '{"name":"Ada","prices":[12,18,5]}'
);
-- {"customer":"Ada","total":35}
```

Construct several output fields without unpacking the document.

06 Return a SQL number

```
SELECT jsonata('$sum(prices)',
  '{"prices":[12,18,5]}'
)::INTEGER AS total;
-- 35
```

Cast the JSON result before joining or doing typed SQL arithmetic.

07 In practice

Pay attention to result shape

JSONata sequences can collapse to a single value. Wrap a projection in array construction when downstream SQL expects JSON[]. The function returns JSON; cast numbers explicitly or use DuckDB JSON extractors for strings.

Full docs & examples ↗ Source ↗